



Advanced 32-Bit Microprocessor 80386

10.1	Salient Features of 80386	10-3
✓	Syllabus Topic : Architecture of 80386 Microprocessor.....	10-5
10.2	80386 Architecture / Functional Block Diagram.....	10-5
10.3	Flag Register of 80386 (EFLAGS – 32 bits)	10-6
	Q. Explain flag register of 80386DX. (Dec. 16, Dec. 18, 5 Marks).....	10-6
	Q. Draw and explain EFLAG register format of 80386 DX. (Dec. 17, 10 Marks).....	10-6
	Q.. Explain VM, RF, IOPL and NT flags of 80386 microprocessor. (May 19, 5 Marks)	10-6
10.3.1	VM : Virtual 8086 Mode.....	10-7
	Q. Explain V86 mode of 80386DX. (May 16, 5 Marks).....	10-7
	Q. Write short note on : Virtual 86 mode of 80386DX (Dec. 16, May 17, May 18, 5 Marks)	10-7
10.3.2	RF : Resume Flag	10-7
10.3.3	NT : Nested Task	10-7
10.3.4	IOPL : I/O Privilege Level.....	10-7
10.3.5	OF : Overflow Flag	10-8
10.3.6	DF : Direction Flag	10-8
10.3.7	IF : Interrupt Enable Flag	10-8
10.3.8	TF : Trap Flag	10-8
10.3.9	SF : Sign Flag	10-8
10.3.10	ZF : Zero Flag	10-8
10.3.11	AC : Auxiliary Carry Flag.....	10-8
10.3.12	PF : Parity Flag	10-9
10.3.13	CF : Carry Flag.....	10-9
✓	Syllabus Topic : 80386 Control Registers.....	10-9
10.4	80386 I Control Registers	10-9
	Q. Write short note on :Control registers of 80386DX (Dec. 16, May 17, 5 Marks).....	10-9
10.4.1	Control Register 0	10-9



10.4.2	Control Register 1 : Not Used (Reserved by Intel)	10-11
10.4.3	Control Registers 2 and 3 : (Used Only For Paging)	10-11
10.4.4	Control Register 2 : (Page Fault Linear Address).....	10-11
10.4.5	Control Register 3 : (Page Directory Base Register - PDBR).....	10-11
✓	Syllabus Topic : Real mode	10-11
10.5	80386 I Real Mode	10-11
	Q. Explain the modes of operation of 80386 microprocessor (Dec. 18, 10 Marks)	10-11
	Q. Differentiate Real Mode, Protected Mode and virtual 8086 mode of 80386 microprocessor. (May 19, 10 Marks)	10-11
10.6	Software Model / Register Model of 80386 in Real Mode.....	10-13
✓	Syllabus Topic : Protected Mode	10-14
10.7	80386 I Protected Mode	10-14
10.8	Protected Mode Register Model / Software Model / Programming Model	10-15
10.9	GDT & GDTR (Global Descriptor Table Register)	10-17
	Q. What is GDT? Explain structure of GDT. (May 17, 5 Marks)	10-17
10.10	IDT & IDTR (Interrupt Descriptor Table Register).....	10-17
10.11	LDT & LDTR (Local Descriptor Table Register).....	10-18
10.12	80386 I Virtual Memory Management.....	10-19
	Q. Explain memory management in details in 80386DX processor. (Dec. 15, 10 Marks)	10-19
	Q. Draw format of selector and explain its field. (Dec. 17, 5 Marks)	10-19
10.13	80386 I Segment Translation.....	10-21
	Q. Explain, with neat diagram, address translation mechanism implemented on 80386 DX. (May 17, 10 Marks)	10-21
	Q. Draw format of selector and explain its field. (Dec. 17, 5 Marks)	10-21
10.14	Descriptor Format.....	10-23
	Q. Explain data segment descriptor with neat diagram. (Dec. 16, 10 Marks)	10-23
	Q. Explain Segment Descriptor of 80386 Processor (May 18, 10 Marks)	10-23
10.15	80386 I Page Translation.....	10-25
	Q. Explain, with neat diagram, address translation mechanism implemented on 80386 DX. (May 17, 10 Marks)	10-25
	Q. Draw format of selector and explain its field. (Dec. 17, 5 Marks)	10-25
10.16	University Questions and Answers	10-28
•	Chapter ends	10-29



10.1 Salient Features of 80386

Address Bus

80386 has a “32 bit” address bus.

- This means it can access a total of $2^{32} = 4\text{GB}$ of physical memory.
- The memory has an address range of 0000 0000H ... FFFF FFFFH.

Memory Address	Data
0000 0000 h	8-bit
0000 0001 h	8-bit
0000 0002 h	8-bit
0000 0003 h	8-bit
---	---
FFFF FFFF h	8-bit

- Though the total address bus is of 32 bits, only the higher 30 bits from $A_{31} - A_2$ are released by the μP .
- The lower 2 lines A_1 and A_0 are used internally by the μP to produce the four bank-enable signals $\overline{\text{BE}}_3 \dots \overline{\text{BE}}_0$.

Data Bus

- 80386 has a “32-bit” data bus. This means 80386 can transfer 32-bit data at a time.
- It also has a 32-bit ALU, which means 80386 can operate on 32-bit numbers in one cycle.
- Hence 80386 is called a “32-bit μP ”.
- 32-bit data is stored in 4 consecutive locations.
- To transfer 32-bit data in one operation 80386 memory is divided into 4 banks of 1 GB each. The banks are enabled by 4 bank-enable signals: $\overline{\text{BE}}_3 \dots \overline{\text{BE}}_0$ produced by the μP .

Address Pipelining

- 80386 performs address pipelining, by putting address of the next machine cycle on the address bus, during T2 state of the current machine cycle.
- This makes the decoder delay transparent and is especially useful for interfacing slower devices as it reduces the number of wait states.

**Virtual Memory**

- 80386 supports Virtual Memory which is implemented using Segmentation and Paging.
- It can access a total Virtual Memory of 64 TB (2^{46}).

Protection

80386 use a protected model for accessing both memory and I/O. It uses 4 Privilege Levels.

Multitasking

- 80386 allows multitasking using timesharing.
- Here several tasks can execute simultaneously by taking a small time slice of the μP . this gives higher system performance.

I/O Addressing

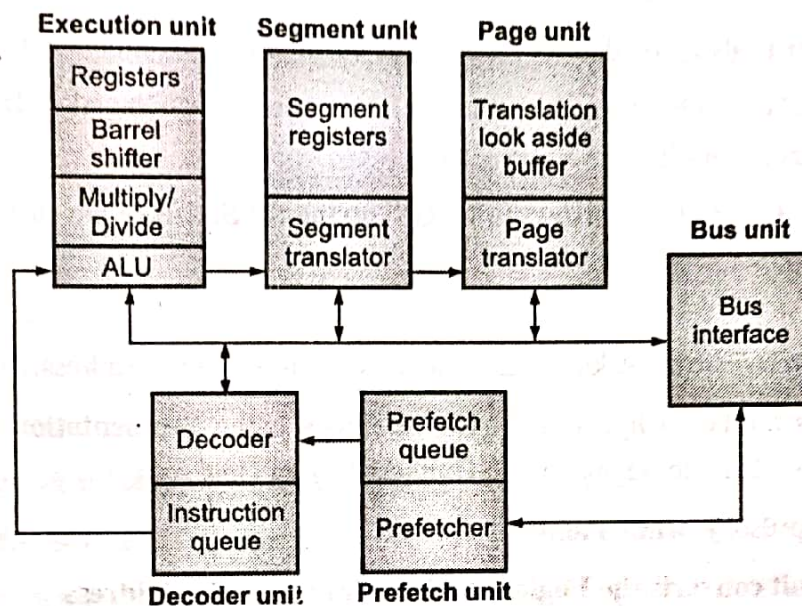
80386 uses a 16-bit I/O address and hence can access up to 2^{16} i.e. 65536 I/O devices with address 0000 h ... FFFF h.

Sr. No.	80386 DX	80386 SX
1.	80386 DX has a 32 bit data bus.	80386 SX has a 16 bit data bus.
2.	Due to 32 bit data bus, the execution speed is higher. Hence the name: DX – Double Execution speed.	Due to 16 bit data bus, the execution speed is lower. Hence the name: SX – Single Execution speed.
3.	32-bit transfers require 4 Memory Banks.	16-bit transfers require 2 Memory Banks.
4.	Has 4 Bank enable signals: $\overline{BE}_3$, $\overline{BE}_2$, $\overline{BE}_1$, $\overline{BE}_0$.	Has only 2 Bank enable signals: $\overline{BHE}$ and $\overline{BLE}$.
5.	4 Bytes are fetched at once in the pipelining queue.	2 Bytes are fetched at a time in the pipelining queue.
6.	Has dynamic data bus sizing of 16-bit and 32-bit data bus, using $\overline{BS16}$ signal.	No such option available as the data bus is only of 16-bits. Hence $\overline{BS16}$ signal not useful.
7.	Used for high performance.	Used for low cost memory and I/O system design.
8.	Comes in a 132-pin ceramic PGA (Pin Grid Array) package for higher performance.	Comes in 100 lead plastic quad flat packages (PQFP) to permit lower cost.



✓ Syllabus Topic : Architecture of 80386 Microprocessor

➤ 10.2 80386 Architecture / Functional Block Diagram



(1F1) Fig. 10.2.1 : Functional Block Diagram of 80386

80386 architecture is divided into 5 independent units.

☞ Bus Unit (Bus Interface Unit)

1. The Bus unit is responsible for transferring data in and out of the μP .
2. It is connected to the external memory and I/O devices, using the system bus.
3. It gets requests from Prefetch unit for fetching instructions and from execution unit for transferring data.
4. If both requests occur simultaneously preference is given to execution unit.

☞ Prefetch Unit

1. The Pre-fetch unit fetches further instructions in advance to implement pipelining.
2. It fetches the next 16 bytes of the program and stores it into the Prefetch Queue.
3. It refills the queue when at least 4 bytes are empty as 80386 has a 32 bit data bus.
4. During a branch, the instructions in the queue are invalid and hence are discarded.

☞ Decode Unit

1. 80386 μP has a separate unit for decoding instructions called the Decode Unit.
2. It decodes the next three instructions and keeps them ready in the Decode Queue.
3. The decoded instructions are stored in Micro-Coded form.
4. During a branch, the instructions in the queue are invalid and hence are discarded.



Execution Unit

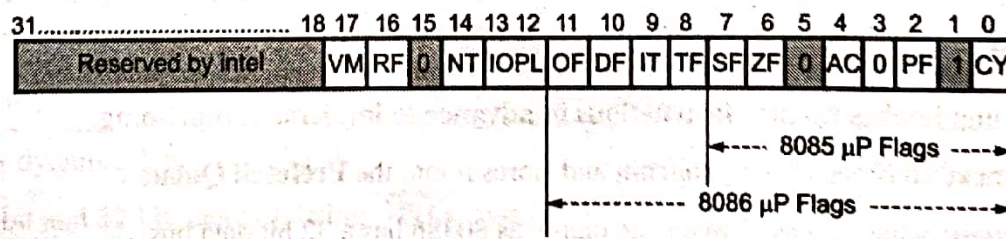
1. Execution Unit performs the main task of **executing instructions**.
2. Normally, execution requires Arithmetic or Logic operations performed by a **32-bit ALU**.
3. It also has dedicated circuits for **32-bit multiplication and division**.
4. A **64-bit barrel shifter** is also provided for faster shifts during multiplication and division.
5. **Operands** for the ALU can either be provided in the **instruction**, or can be taken **from memory** or could be taken from the **32-bit registers** like EAX, EBX etc.
6. Additionally there is a **32-bit Flag register (EFLAGS)** giving the **Status** of the current result.

Memory Unit

1. The Memory unit converts Virtual Address (Logical address) to Physical Address.
2. 80386 μ P implements **64 Terra bytes of Virtual memory** using **Segmentation and Paging**. Hence the Memory Unit is sub-divided into Segmentation Unit and Paging Unit.
3. **Segmentation is compulsory, while Paging is optional**.
4. **The Segmentation Unit converts the Logical Address into a Linear Address**.
5. **The Paging Unit converts the Linear Address into a Physical Address**.
6. If Paging is not used, then the Linear Address itself is the Physical Address.

10.3 Flag Register of 80386 (EFLAGS – 32 bits)

- Q.** Explain flag register of 80386DX. (Dec. 16, Dec. 18, 5 Marks)
- Q.** Draw and explain EFLAG register format of 80386 DX. (Dec. 17, 10 Marks)
- Q..** Explain VM, RF, IOPL and NT flags of 80386 microprocessor. (May 19, 5 Marks)



(1F2)Fig. 10.3.1 : E Flag Register of 80386

- As seen from the above diagram, the lower 12 bits (11 ... 0) of EFLAGS are same as those in 8086. These are the only flags available when μ P is in Real Mode.
- The additional 5 flags are only available once μ P enters Protected mode by making PE bit = 1, in CR0 register.



10.3.1 VM : Virtual 8086 Mode

Q. Explain V86 mode of 80386DX. (May 16, 5 Marks)

Q. Write short note on : Virtual 86 mode of 80386DX (Dec. 16, May 17, May 18, 5 Marks)

This flag is used to make 80386 operate in Virtual 8086 Mode (V86).

If VM = 1, enter Virtual 8086 Mode.

V86 mode is basically used to run 8086 programs in a faster environment of 80386 using Multitasking and Protection.

V86 mode can only be entered if μP is working in Protected Mode.

Once in Virtual 8086 Mode, we can return back to Protected Mode by making VM $\leftarrow 0$.

A special program called Virtual 8086 Monitor is responsible for switching back and forth between Protected Mode and Real Mode.

10.3.2 RF : Resume Flag

Resume flag is useful during debugging.

If RF = 1, then any debug fault in the next instruction will be ignored. RF is automatically reset after the next instruction.

In 80386 μP , some fault handlers (ISRs) return back to the same instruction that caused the fault instead of returning back to the next instruction. By keeping RF = 1, we ensure that the program resumes after such a fault instead of repeatedly generating breakpoint faults on the same instruction.

10.3.3 NT : Nested Task

NT flag is used to indicate that the current task is nested i.e. it is invoked by another task.

If NT = 1 then, the current task is Nested and its TSS (Task State Segment) has a valid "back link" to the TSS of the previous task.

This bit is automatically set whenever a Nested task is initiated, and thereafter can only be reset by software.

NT flag is checked by the IRET instruction to know whether it should perform an "Intra-task" return or an "Inter-task" return.

10.3.4 IOPL : I/O Privilege Level

These bits are used to assign I/O Privilege Level.

80386 μP has four privilege levels used for protection mechanism.

Privilege Level = 0 is the highest Privilege Level and 3 is the lowest.

IOPL bits define the numerically maximum Privilege Level (logically lowest) at which a task must be running to access I/O devices.

If IOPL bits = 00 then only highest privileged tasks running at PL=0 can perform I/O instructions.

If IOPL bits = 11 then all tasks at any Privilege Level can perform I/O instructions.

10.3.5 OF : Overflow Flag

It is used to indicate an overflow during a signed operation.

OF is determined as an Ex-Or between the last and the second last carry.

For an 8-bit operation, $OF = C_7 \text{ Ex-Or } C_6$.

10.3.6 DF : Direction Flag

It is used to give the direction during String operations.

If $DF = 1$ then, auto decrement. If $DF = 0$ then, auto increment.

10.3.7 IF : Interrupt Enable Flag

It is used to enable the hardware interrupt INTR.

When $IF = 1$, INTR is enabled. When $IF = 0$, INTR is disabled.

10.3.8 TF : Trap Flag

It is used to perform Single Stepping.

If $TF = 1$ then perform Single Stepping.

If $TF = 1$ then stop/ do not perform Single Stepping.

10.3.9 SF : Sign Flag

It is used to indicate the MSB of the result.

If $SF = 1$ then, MSB of the result is 1, so the number is treated as -ve.

If $SF = 0$ then, MSB of the result is 0, so the number is treated as +ve.

10.3.10 ZF : Zero Flag

It is used indicate if the result has become zero.

If $ZF = 1$ then the result is zero.

If $ZF = 0$ then, the result is non-zero.

10.3.11 AC : Auxllary Carry Flag

It indicates a Carry from the lower nibble to the higher nibble.

If $AC = 1$ then, there was a carry from Lower Nibble to Higher Nibble.

If $AC = 0$ then, there was no carry from Lower Nibble to Higher Nibble.

Even during a 16-bit or a 32-bit operation AC only indicated the carry from the lowest nibble to the next nibble.

**10.3.12 PF : Parity Flag**

Indicates whether the result has even or odd parity.

If PF = 1 then, result has even parity.

If PF = 0 then, result has odd parity.

10.3.13 CF : Carry Flag

It indicates if a carry was produced beyond the MSB of the result.

If CF = 1 then, Carry produced beyond the MSB of the result.

If CF = 0 then, Carry not produced beyond the MSB of the result.

✓ Syllabus Topic : 80386 Control Registers**10.4 80386 | Control Registers**

Q. Write short note on :Control registers of 80386DX (Dec. 16, May 17, 5 Marks)

There are four; 32-bit Control registers in 80386.

MSW																				
P G	0000000000000000								000000000000								E T	S M	E M P E	CR0
	Not used																			
Page fault linear address																CR2				
Page directory base												000000000000				CR3				

**ET : Extension Type**

This bit is used to indicate the type of Math Co-Processor used with 80386.

If ET = 1, then 80387 Math Co-Processor is used.

If ET = 0, then 80287 Math Co-Processor is used.

Note : This bit was reserved (R) in the initial versions of 80386 and was introduced later once 80387 was developed.

TS : Task Switched

- This bit is made "1" to indicate if a Task Switch is performed.
- 80386 μ P implements "Multitasking", and thus it switches between various tasks, giving the programmer the impression that all tasks are running concurrently.
- This significantly improves the overall system performance.
- If TS = 1, it means a task switch is performed. Now the TSS of the current task has a back-link to the previous task.

Note : If TS bit = 1, then a type 7 interrupt called Math Co-Processor not available is caused, whenever a coprocessor instruction is encountered.

EM : Emulate Coprocessor

- This bit is made "1" in the absence of a Math Co-Processor so that if a coprocessor instruction is encountered, then it will be executed by an on chip emulator.
- If this bit is 0, then the coprocessor instruction will be executed by 80387/80287 whichever is present in the system.

MP : Math Co-processor Present

This bit is made "1" to indicate that a Math Co-Processor like 80387 or 80287 is present.

Note : Out of EM bit and MP bit, only one of them must be "1".

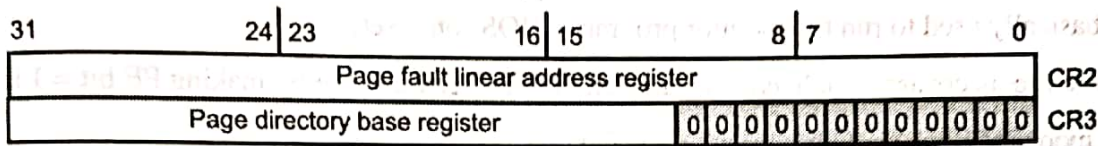
PE : Protection Enable

- This bit is made "1" to enter protected mode.
- On reset, by default this bit is "0". It is the only bit of CR0 which is also available in Real Mode.



10.4.2 Control Register 1 : Not Used (Reserved by Intel)

10.4.3 Control Registers 2 and 3 : (Used Only For Paging)



(1F4)Fig. 10.4.3 : CR2 & CR3

10.4.4 Control Register 2 : (Page Fault Linear Address)

This is a 32-bit register giving the Linear Address that caused the last Page fault. A page Fault occurs when the desired page is not present in the Physical Memory.

10.4.5 Control Register 3 : (Page Directory Base Register - PDBR)

- 80386 μ P implements 2-level Page translation mechanism (explained in later sections). Information about the various pages is stored in various page tables.
- Addresses of these page tables are stored in the page directory. CR3 gives the base address (starting address) of the Page Directory.
- Note that PDBR is only 20 bits. It just gives the upper 20-bits of the starting address of the page directory. That's because the page directory is of 4KB and is stored at a 4KB aligned location, so the last 12 bits of the starting address are assumed to be "0".

Operating modes of 80386

80386 can operate in three different modes.

1. Real Mode
2. Protected Mode
3. Virtual 8086 Mode

✓ Syllabus Topic : Real mode

10.5 80386 | Real Mode

Q. Explain the modes of operation of 80386 microprocessor (Dec. 18, 10 Marks)

Q. Differentiate Real Mode, Protected Mode and virtual 8086 mode of 80386 microprocessor. (May 19, 10 Marks)

- It is the **default mode selected when 80386 is reset**. In this mode, 80386 μ P simply behaves as a **fast 8086 machine**.
- All registers are just like 8086. Even the **memory used is only 1 MB**, just like in 8086. Physical address calculation is also like in 8086:



Physical Address = Base Address $\times$ 10H + Offset Address

Eg: Segment address (Base address) = 5142H and offset address

= 0006H then the Physical address will be = 51426H

This mode is basically used to run the monitor program (BIOS) on reset.

Once the required registers are initialized, we can switch to Protected mode by making PE bit = 1 in CR0.

The Software model of Real Mode is shown on the next page.

Segment Registers

- There are 6, 16-bit segment registers containing the base addresses of their respective segments.
- The registers are CS, SS, DS, ES, FS and GS.

Offset Registers

- There are 5, 16-bit offset registers containing the offset addresses for various segments.
- The registers are IP, SP, BP SI and DI.
- The 32-bit extended form (ESI, EDI etc) of these registers is not used in Real Mode.

Data Registers

- There are 4, 16-bit data registers used as General Purpose Registers during programming.
- The registers are AX, BX CX and DX. They can be also used as 8, 8-bit registers AL, AH, BL, BH, CL, CH, DL and DH.
- The 32-bit extended form (EAX, EBX etc) of these registers is not used in Real Mode.

Flags

- Only the lower 12-bits of the Flag Register are used in the Real Mode.
- The 32-bit extended form of Flag Register i.e. EFLAGS is available only in Protected Mode.

Control Registers

- Only the LSB of CR0 is available in Real Mode.
- The remaining bits of CR0 and the remaining Control Registers are available only in Protected Mode.
- The LSB of CR0 is "PE" bit which should be made = "1" to begin Protected Mode.

Debug Registers and Test Registers : These registers are not available in Real Mode.

Memory Range

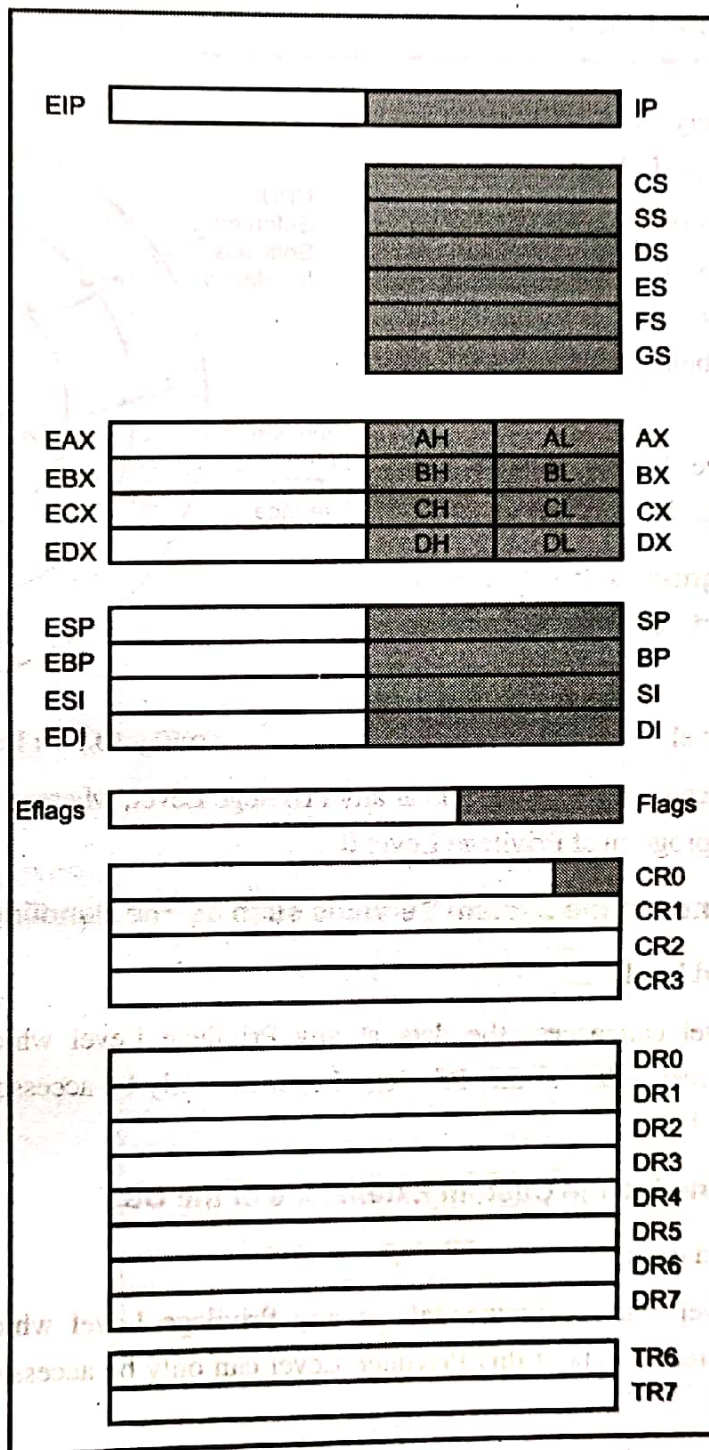
The size of memory available in real mode is 1 MB and has address range from 00000H ... FFFFFH, just like in 8086.

I/O Range

A total of 64K I/O addresses are available having a range from 0000H ... FFFFH, just like in 8086.



10.6 Software Model / Register Model of 80386 In Real Mode



(1F5) Fig. 10.6.1 : Software Model In Real Mode



✓ Syllabus Topic : Protected Mode

➤ 10.7 80386 | Protected Mode

- 80386 μ P provides a very advanced mode of operations called the Protected Mode.
- In Protected Mode, 80386 μ P provides dedicated hardware to prevent user programs from affecting other user programs and also safeguards the Operating System from being affected by user programs.
- There are Four Privilege Levels, assigned to programs and data to define their privileges.

Level 0 : This level is assigned to the Operating System Kernel (Main part of the Operating System).

- It is the most privileged level.
- Any program at this level can access all the data at any Privilege Level, whereas a data at this Privilege Level can only be accessed by a program at Privilege Level 0.

Level 1 : This level is assigned to the System Services such as File Handling, Device Drivers.

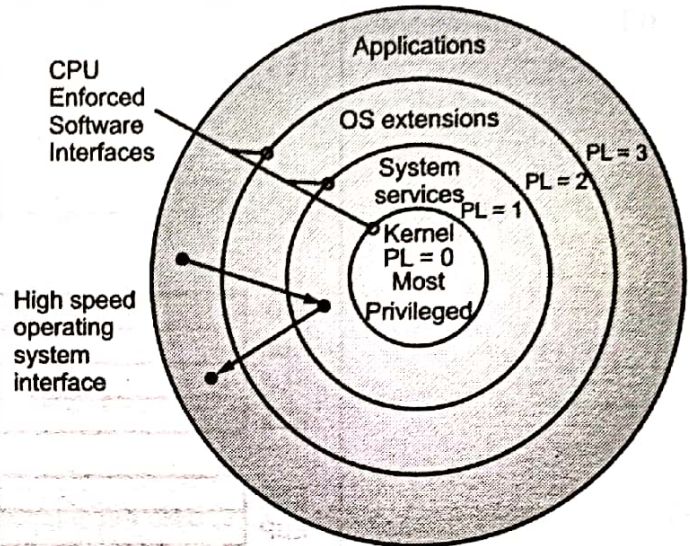
- It is the 2nd most privileged level.
- Any program at this level can access the data at any Privilege Level which is lower than this level (numerically higher), whereas a data at this Privilege Level can only be accessed by a program at Privilege Level 0 or Privilege Level 1.

Level 2 : This level is assigned to the Custom Extensions of the OS.

- It is the 3rd most privileged level.
- Any program at this level can access the data at any Privilege Level which is lower than this level (numerically higher), whereas a data at this Privilege Level can only be accessed by a program at Privilege Level 0, 1 or 2.

Level 3 : This level is assigned to all the User Application and Programs.

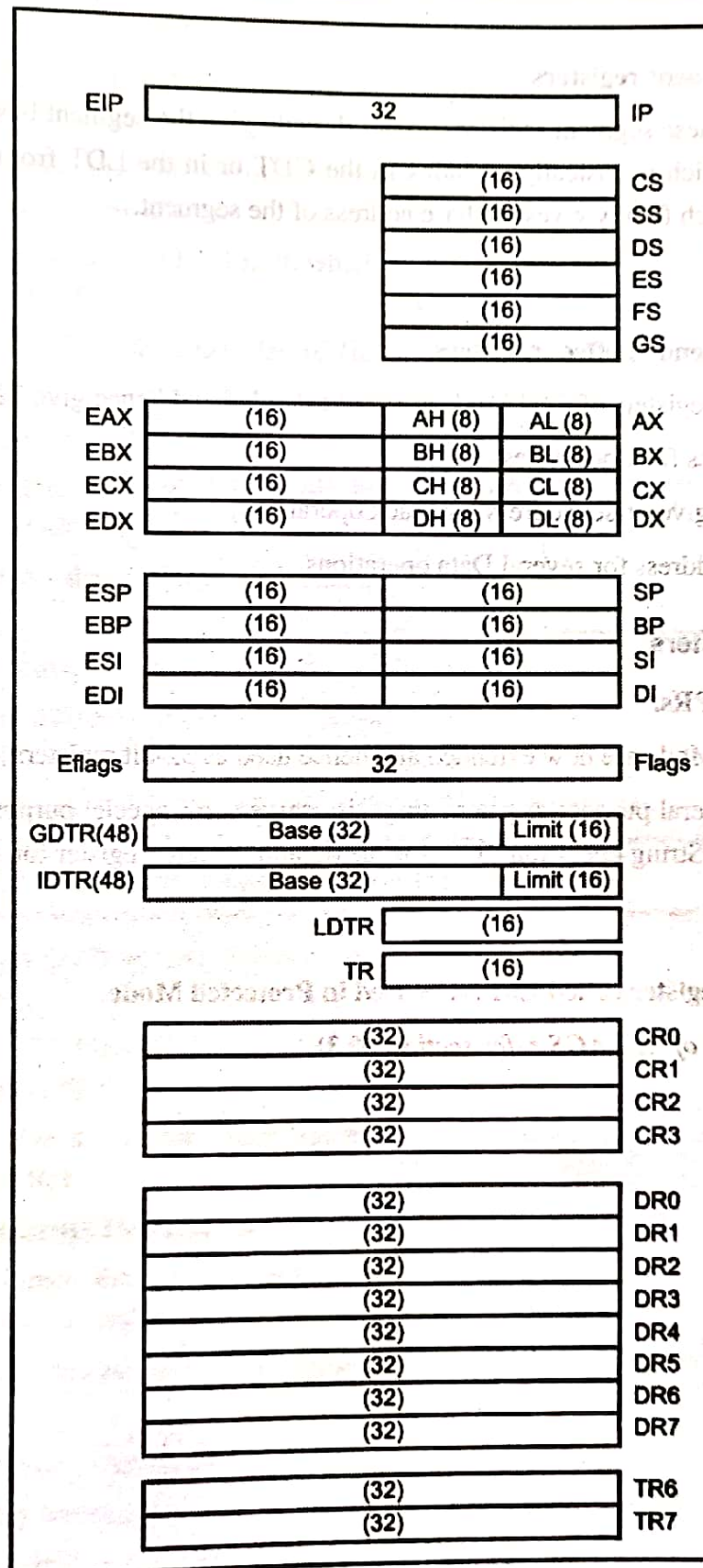
- It is the least privileged level.
- Any program at this level can normally access the data at Privilege Level 3, whereas a data at this Privilege Level can be accessed by a program at any Privilege Level 0...3.



(116) Fig. 10.7.1 : Privilege levels



10.8 Protected Mode Register Model / Software Model / Programming Model



(17) Fig. 10.8.1 : Software Model in Protected Mode



The Software Model of 80386 is explained as follows :

☞ Segment Registers

There are six, 16-bit segment registers.

Unlike Real mode, now these segment registers do not directly give the segment base address. Instead, these registers give a "Selector" which is basically an index in the GDT or in the LDT from which the descriptor is loaded. It is the descriptor which finally gives the base address of the segment.

☞ Offset Registers

- There are five, 32-bit extended offset registers.
- The 16-bit offset address registers of Real Mode are now extended and hence give 32 bit offset addresses.
- EIP gives the offset address for Code access.
- ESP and EBP are used to give offset address for Stack operations.
- ESI and EDI give offset address for several Data operations.

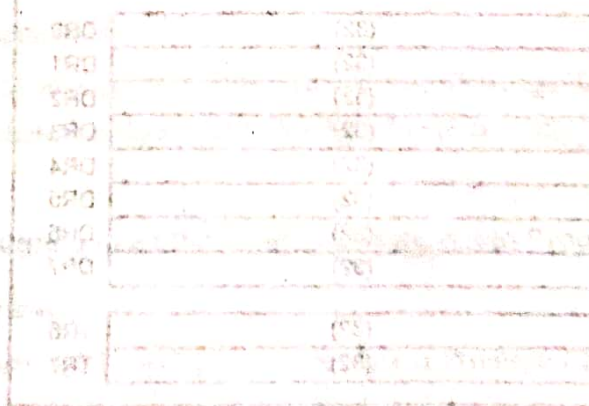
☞ General Purpose Registers

- There are four, 32-bit GPRs.
- The 16-bit GPRs of Real Mode are now extended and hence used as 32-bit registers.
- Besides being used as general purpose registers, they also serve some special purposes like, AX – acts as an accumulator in MUL/DIV/String operations, CX – is the default "count" register for several instructions etc.

☞ Flag Register

80386 has a 32-bit flag register called EFLAGS used in Protected Mode.

(For diagram and answer of EFLAGS refer section 10.3)



10.9 GDT & GDTR (Global Descriptor Table Register)

Q. What is GDT? Explain structure of GDT. (May 17, 5 Marks)

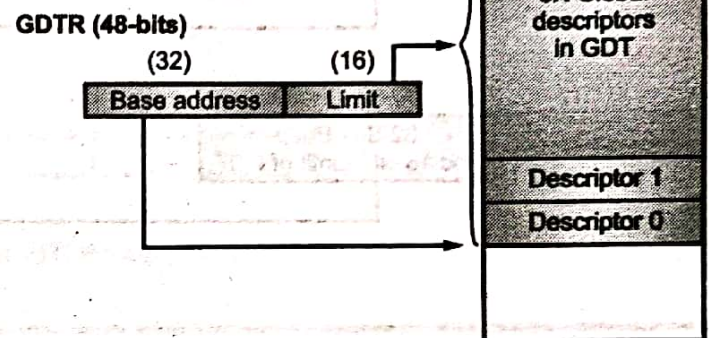
1. GDTR is a 48 bit register. It defines the GDT in the Physical Memory.
2. The upper 4 bytes (32 bits) give the starting address of the GDT also called the base address.
3. The lower 2 bytes give the 16-bit Limit which decides the size of the GDT.
4. As the Limit field is 16-bits, the max size of the GDT can be 64 KB.

The size of GDT is always Limit + 1.

If Limit is FFFFH then size of GDT will be $65535+1=65536$ i.e. 64KB.

5. The GDT contains descriptors of Global Segments.
6. Each descriptor is of 8 bytes.
It gives the Starting Address, Limit and Access Rights of the segment

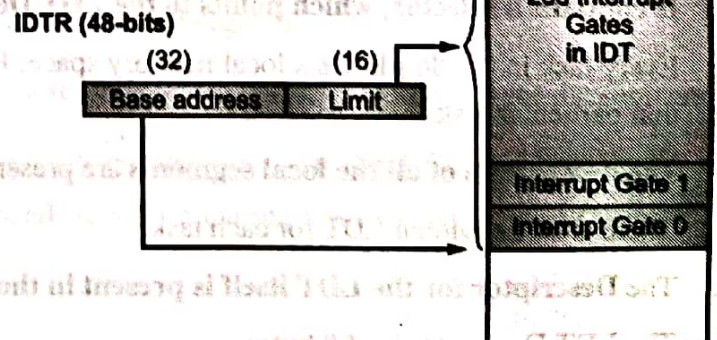
7. The GDT can contain max $64KB/8 = 8K$ descriptors (8192 Descriptors).



(1F8) Fig. 10.9.1 : GDT and GDTR

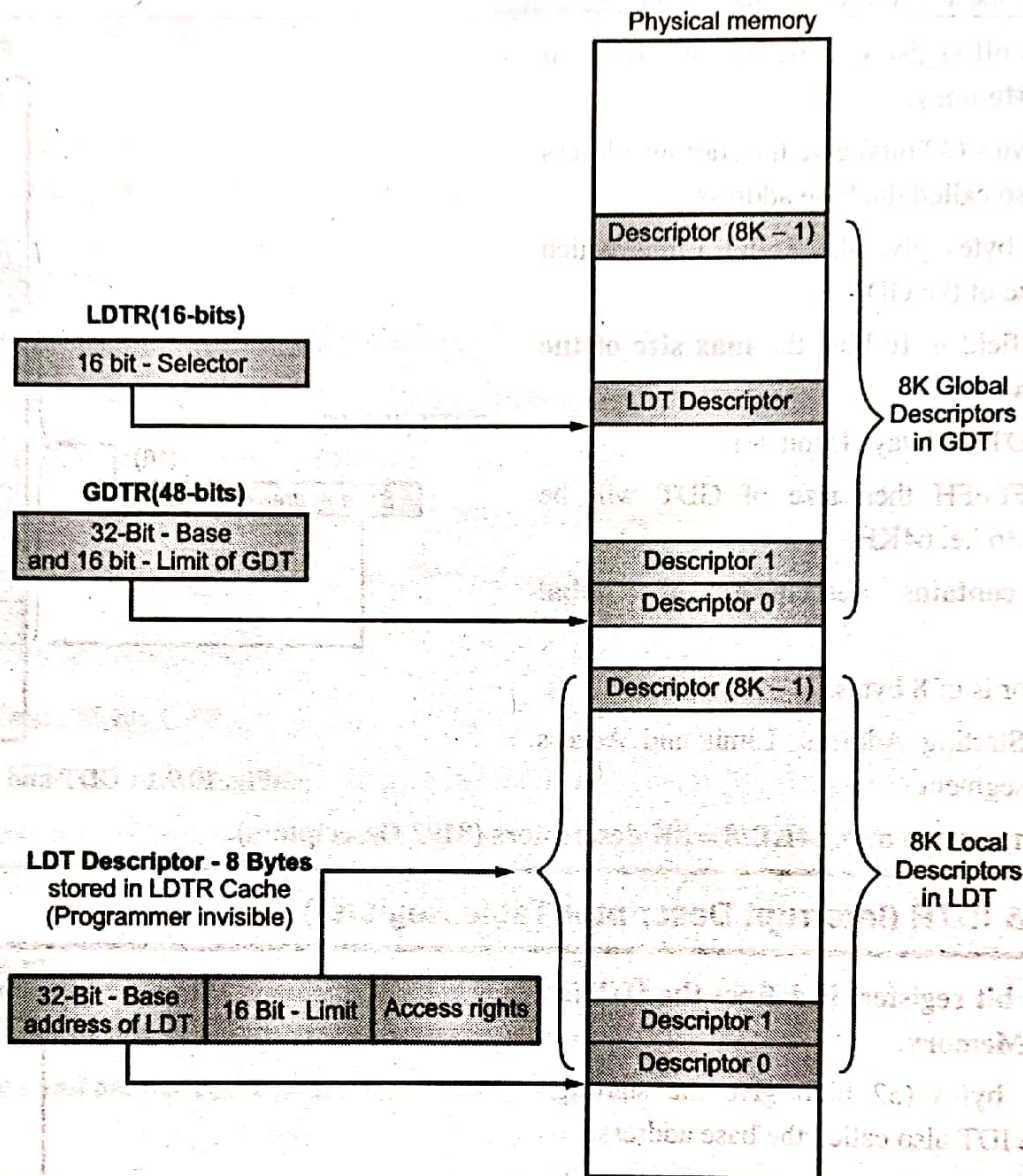
10.10 IDT & IDTR (Interrupt Descriptor Table Register)

1. IDTR is a 48 bit register. It defines the IDT in the Physical Memory.
2. The upper 4 bytes (32 bits) give the starting address of the IDT also called the base address.
3. The lower 2 bytes give the 16-bit Limit which decides the size of the IDT.
4. The IDT contains Interrupt Descriptors.
5. These Descriptors direct the μP towards the respective ISR whenever an interrupt occurs.
Hence the Descriptors are also called "Interrupt Gates".
6. Each Interrupt Descriptor is of 8 bytes.
7. 80386 μP supports 256 Interrupts.
8. Hence max size of the IDT is $256 \times 8 = 2KB$.
9. The size of IDT is Limit + 1.
10. Hence max value of Limit must be 07FFH.



(1F9) Fig. 10.10.1 : IDT & IDTR

10.11 LDT & LDTR (Local Descriptor Table Register)



(1F10) Fig. 10.11.1 : LDT & LDTR

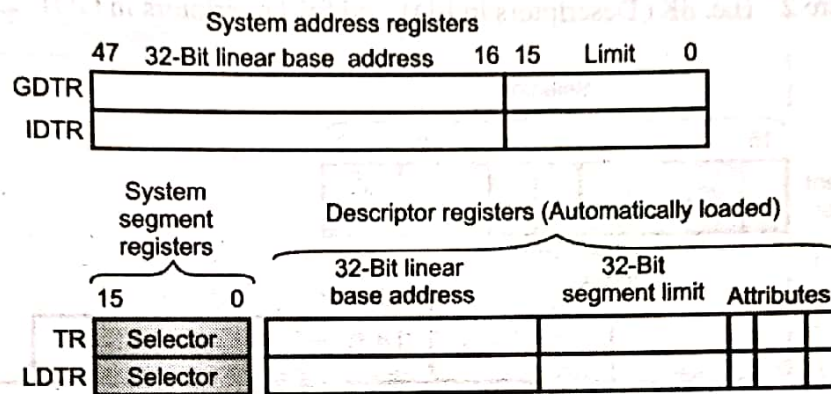
1. LDTR is a 16 bit register.
2. It contains a selector, which points to the LDT Descriptor in the GDT.
3. Every task in 80386 μ P has a local memory space. Here it has its local segments, which are only available to that particular task.
4. The descriptors of all the local segments are present in the LDT of that task.
5. There is an individual LDT for each task.
6. The Descriptor for the LDT itself is present in the GDT.
7. The LDT Descriptor is of 8 bytes.
8. It contains a 32 bit base address of the LDT, 16 bit Limit and Access rights information.



9. Once we load a new selector in the LDTR, the corresponding LDT Descriptor is fetched from the GDT and loaded into a LDT Descriptor Cache and thus it provides the address and limit of the LDT.
10. The structure of LDT is similar to that of GDT.

It also has max 8K Descriptors each of size 8 Bytes and hence **max size of LDT is also 64KB.**

Summary



(1F11) Fig. 10.11.2 : GDTR, IDTR and LDTR

10.12 80386 | Virtual Memory Management

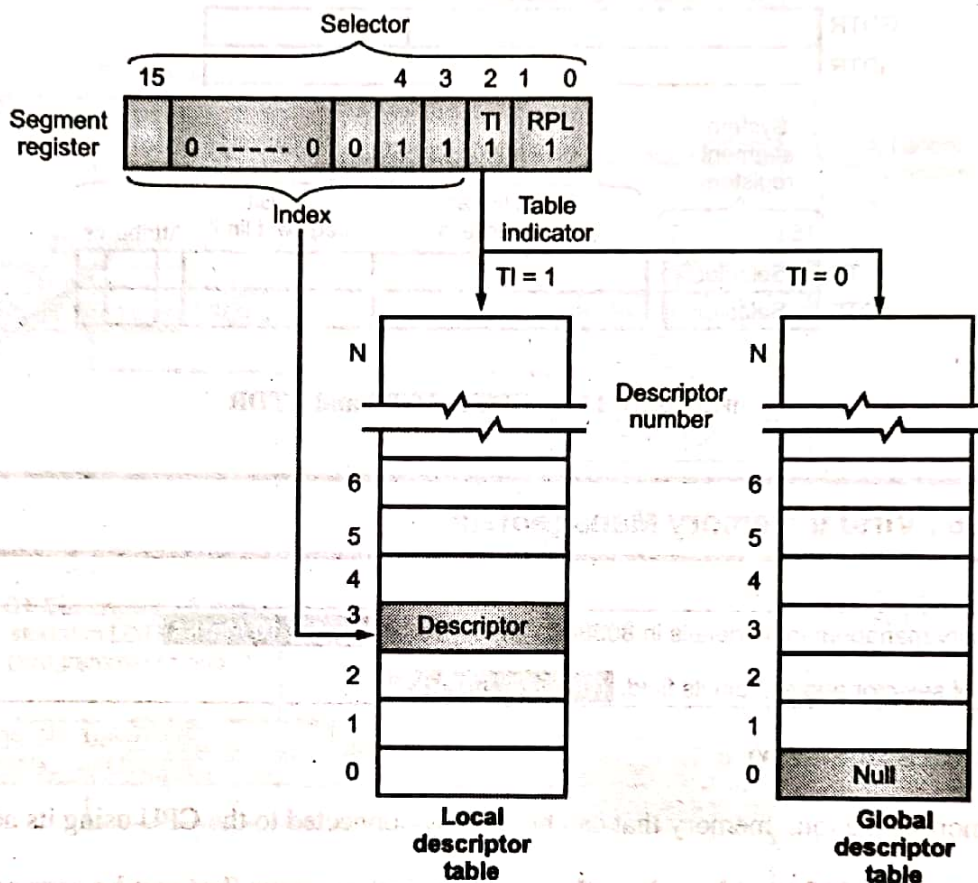
- Q. Explain memory management in details in 80386DX processor. (Dec. 15, 10 Marks)
- Q. Draw format of selector and explain its field. (Dec. 17, 5 Marks)

Virtual and Physical Address Space

1. Physical Memory is the total memory that can be directly connected to the CPU using its address bus.
2. Since 80386 μ P has a 32 bit address bus, the total physical memory that can be connected is $2^{32} = 4GB$.
3. Virtual Memory is the total memory space that can be addressed by the CPU registers.
4. Virtual address is basically the combination of a 16-bit segment register and a 32 bit offset register.
5. In Real Mode, the segment register gives the starting address of the segment.
6. But in Protected Mode, the segment register just gives a selector which selects a Descriptor for the segment.
7. Though the selector is of 16-bits, only 14 bits are used as two bits give the Privilege Level used for protection as seen above. Each selector value corresponds to a different segment. Hence there can be max 2^{14} segments.



8. The locations within the segment are identified by their offset addresses. Since offset addresses are 32 bit, each segment can be max $2^{32} = 4\text{GB}$. Hence the max total Virtual memory that can be accessed = Max number of segments $\times$ Max size of each segment $= 2^{14} \times 2^{32} = 2^{46} = 2^6 \times 2^{40} = 64 \times 1\text{TB} = 64 \text{ Terra Bytes}$.
9. Out of 14 bits of the selector, one bit is used as a table identifier (TI) if TI = 1 then use LDT, if TI = 0 then use GDT.
10. So basically there are 2^{13} (i.e. 8K) Descriptors in LDT and 8K Descriptors in GDT.



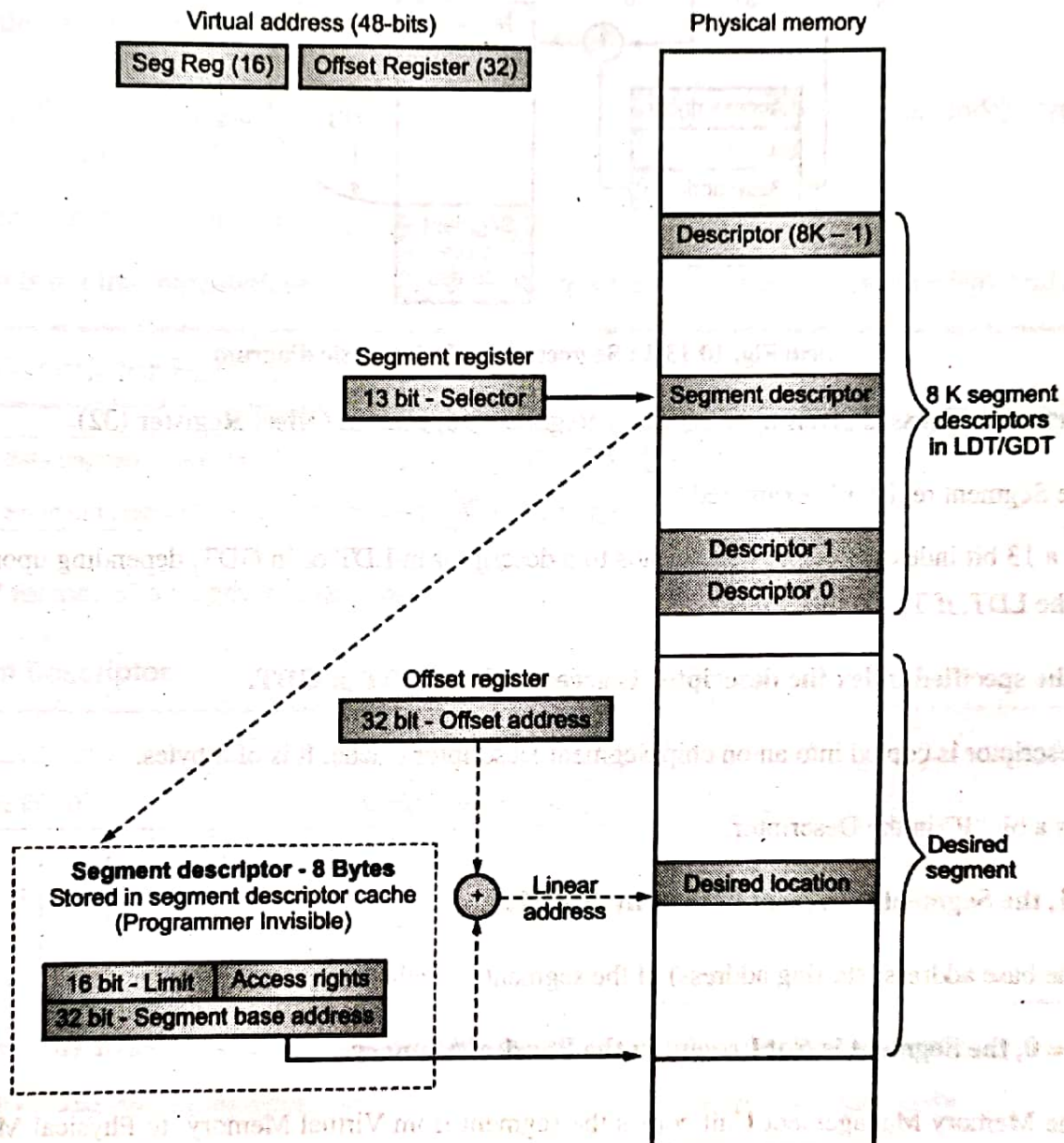
(1F12) Fig. 10.12.1 : Working of Segment Selector

11. This means the total 64TB Virtual space is divided into 32TB of Global space and 32 TB of Local Space.
12. The total 48 bit address having 16 bit segment address and 32 bit offset address is called Virtual address. It is converted into a 32 bit physical address using two translations; Segment translation (compulsory) and page translation (optional).
13. Segment translation converts 48-bit Virtual address into 32 bit Linear Address which is further converted into a 32 bit Physical Address by Page translation.



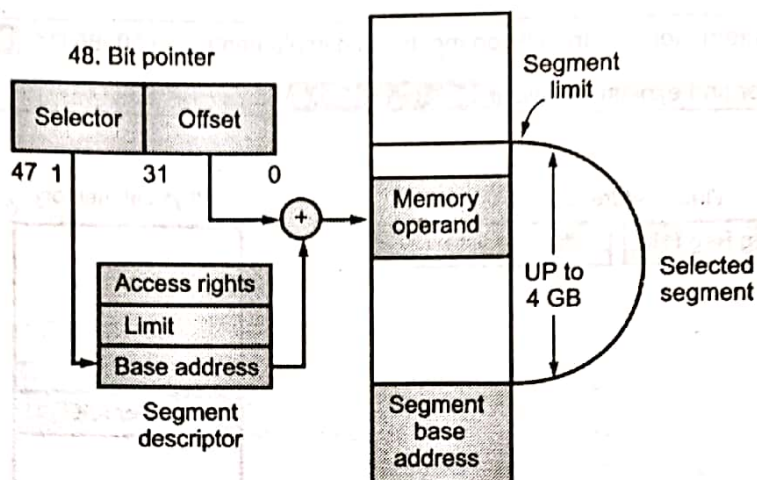
10.13 80386 | Segment Translation

- Q. Explain, with neat diagram, address translation mechanism implemented on 80386 DX. (May 17, 10 Marks)
- Q. Draw format of selector and explain its field. (Dec. 17, 5 Marks)



(1F13) Fig. 10.13.1 : Segment translation

Basic diagram



(1F14) Fig. 10.13.2 : Segment translation basic diagram

1. The Virtual Address is given by a Segment Register (16) and an Offset Register (32).
2. First the Segment register is examined.
It gives a 13 bit index (selector) which leads to a descriptor in LDT or in GDT, depending upon the TI bit. If TI = 1 the LDT, if TI = 0 then GDT.
3. Using the specified index the descriptor is accessed from LDT of GDT.
4. This Descriptor is copied into an on chip Segment Descriptor Cache. It is of 8 bytes.
5. There is a bit "P" in the Descriptor.
6. If P = 1, the Segment is Present in the Physical Memory.
7. Then the base address (starting address) of the segment is available in the Descriptor.
8. If "P" = 0, the Segment is Not Present in the Physical Memory.
9. Now the Memory Management Unit copies the segment from Virtual Memory to Physical Memory, makes the "P" bit = 1 in the Descriptor and stores its base address into the Descriptor for further use.
10. To this base address, the 32-bit offset address is added.
11. This finally gives the 32 bit Linear Address of the desired location.
12. For subsequent access to the same segment, the Descriptor which is already cached is used.



13. The "P" bit will be 1.
14. So the base address will be directly obtained from the Descriptor and the offset address will be added to it to produce the Linear Address.
15. Before granting access to the desired location, a protection check is performed based on the information given by the access rights in the Descriptor.
16. Several types of checks such as limit not exceeded, type of segment (data or code), type of access (read/write/execute), privilege level etc carried out.
If the checks are valid then access is granted, else a general protection fault occurs.
17. If Paging is not implemented, then the 32 bit linear address is the final 32 bit Physical Address.

10.14 Descriptor Format

- Q. Explain data segment descriptor with neat diagram. (Dec. 16, 10 Marks)
- Q. Explain Segment Descriptor of 80386 Processor (May 18, 10 Marks)

General format of a Segment Descriptor

Segment Descriptor

31						0	
SEGMENT BASE 15 ... 0						SEGMENT LIMIT 15...0	
BASE 31...24	G	D/B	0	AVL	LIMIT 19 ... 16	ACCESS RIGHTS BYTE	BASE 23 ... 16

D/B 1 = Default Instruction Attributes are 32-Bits G Granularity Bit 1 = Segment length is page granular
 0 = Default Instruction Attributes are 16-Bits 0 = Segment length is byte granular
 AVL Available field for user or OS 0 Bit must be zero (0) for compatibility with future processors

Note : In a maximum-size segment (i.e. a segment with G = 1 and segment limit 19...0 = FFFFFH), the lowest 12 bits of the segment base should be zero (i.e. segment base 11.....000 = 000H).



Access Rights Byte Definition for Code and Data Descriptions

Bit position	Name	Function	
7	Present (P)	P = 1 Segment is mapped into physical memory. P = 0 No mapping to physical memory exist , base and limit are not used.	
6-5	Descriptor privilege Level (DPL)	Segment privilege attribute used in privilege tests.	
4	Segment descriptor (S)	S = 1 Code or Data (includes stacks) segment descriptor S = 0 System segment descriptor or gate descriptor	
3	Executable (E)	E = 0 Descriptor type is data segment :	} If data segment (S = 1, E = 0)
2	Expansion Direction (ED)	ED = 0 Expand up segment, offsets must be $\leq$ limit. ED = 1 Expand down segment, offsets must be $>$ limit.	
1	Writeable (W)	W = 0 Data segment may not be written into. W = 1 Data segment may be written into.	
3	Executable (E)	E = 1 Descriptor type is code segment :	} If code segment (S = 1, E = 1)
2	Conforming (C)	C = 1 Code segment may only be executed when CPL $\geq$ DPL and CPL remains unchanged.	
1	Readable (R)	R = 0 Code segment may not be read R = 1 Code segment may be read.	
0	Accessed (A)	A = 0 Segment has not been accessed A = 1 Segment selector has been loaded into segment register or used by selector test instructions.	

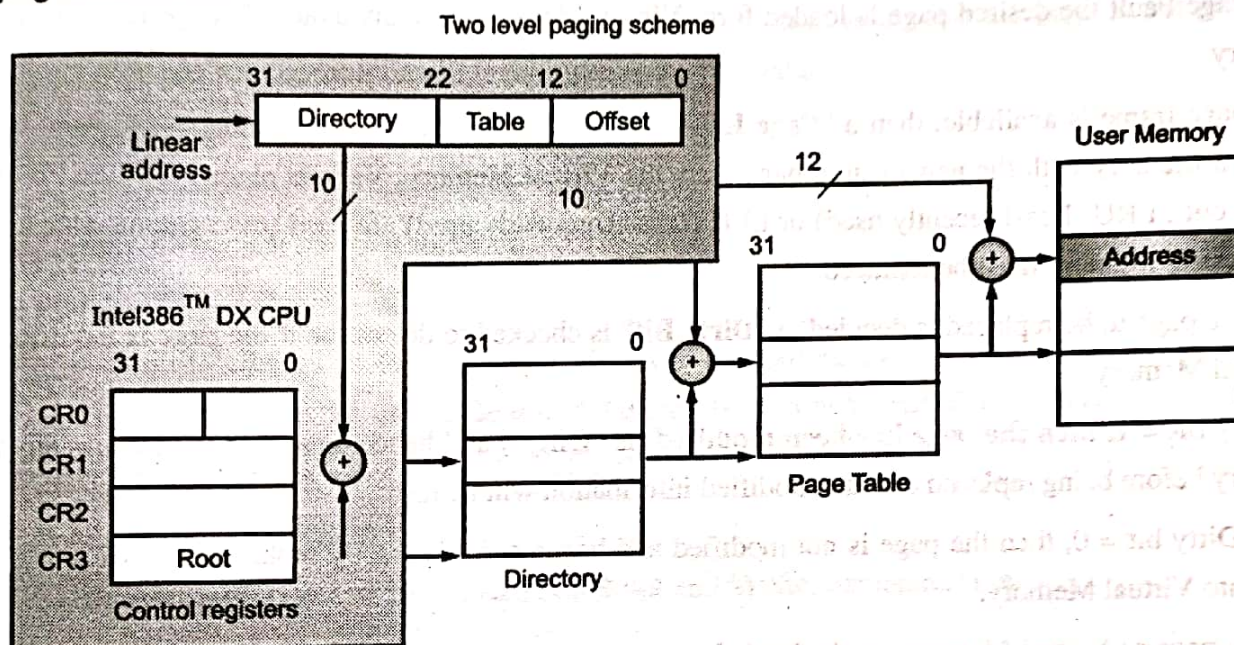
(1F16) Fig. 10.14.1 : Segment descriptor format with access rights byte



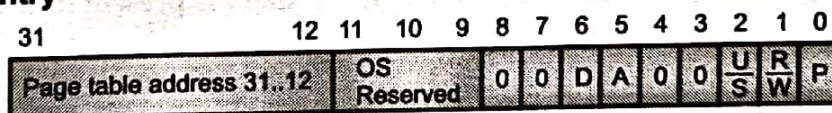
10.15 80386 | Page Translation

- Q. Explain, with neat diagram, address translation mechanism implemented on 80386 DX. (May 17, 10 Marks)
- Q. Draw format of selector and explain its field. (Dec. 17, 5 Marks)

Paging Mechanism

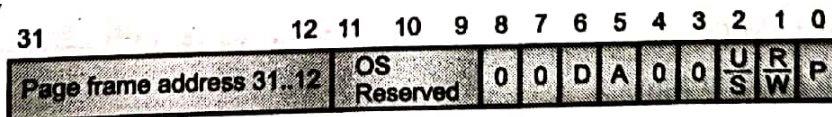


Page Directory Entry



(1F18) Fig. 10.15.2 : Page Directory Entry

Page Table Entry



(1F19) Fig. 10.15.3 : Page Table Entry

- The Virtual Memory space is divided into equal size blocks of 4KB called "Pages".
- The Physical Memory space (also called Main Memory) is also divided into equal size blocks of 4 KB called Page Frames (also simply called pages).
- As Physical Memory is of 4 GB and page size is 4 KB there are total 1 M pages (2^{20}) in the Physical Memory.



4. A page from Virtual Memory is loaded into any available page frame of Physical Memory.
5. Whenever a page is required to be accessed, the μP first checks if the desired page is present in the Physical Memory.
If so, it is called a "HIT" and the operation is performed on the Physical Memory.
6. A "Page Fault" (MISS) occurs when the desired page is not present in the Physical Memory.
7. On a Page Fault the desired page is loaded from Virtual Memory into any available page frame of Physical Memory.
8. If no page frame is available, then a "Page Replacement" is performed by replacing an old page from the Physical Memory with the new desired page from the Virtual Memory. Various algorithms like FIFO (First in first out), LRU (Least recently used) or LFU (Least frequently used) are used to determine which page of the Physical Memory must be replaced.
9. Once the page to be replaced is decided, a "Dirty Bit" is checked to determine if the page is modified in the Physical Memory.
10. If Dirty bit = 1, then the page has been modified (is "Dirty") and hence must be copied back into Virtual Memory before being replaced else the modified information will be lost.
11. If the Dirty bit = 0, then the page is not modified and hence can be directly replaced without being copied back into Virtual Memory.
12. Since a page of Virtual Memory can be loaded into any page frame of Physical Memory, a "Page Table" is required to give the mapping between Virtual Memory page number and Physical Memory page frame number.
13. Simply speaking the Page table tells which page of Virtual Memory is present in which page of Physical Memory.
14. But since there are too many page frames in the Physical Memory (2^{20} i.e. 1M), the page table will become too large and searches will become extremely slow.
15. Hence the mechanism is further subdivided.
16. Instead of having straight 1M (2^{20}) entries in the page table, there are 1K (2^{10}) entries in a page table and there are 1K (2^{10}) such page tables.
17. $\{2^{20} = 2^{10} \times 2^{10} \dots\dots\}$
18. Each page table is of 4KB and has 1K "Page Table Entries" (PTEs) each of size 4 bytes. Each PTE gives information about a Page Frame.
19. The PTE has following information:
20 bit page frame address : Gives the upper 20 bits of the starting address of the page frame. Lower 12 bits are 0...0 as the page is of 4 KB and starts from a
4 KB aligned location (refer Bharat Sir's lecture notes...)



D : Dirty bit indicates whether the page has been modified (1) or not (0).

A : Accessed Bit tells whether the page has been accessed or not (1 means accessed). This is used by replacement algorithms

U/S : User or Supervisor and R/W : Read or Read and Write give protection information

P : Present bit indicates whether the page is present in the Physical Memory.

If $P = 1$ then the page is present and the 20 bit address field is valid, else the page is not present in the Physical Memory and the 20 bit address field is obviously invalid.

20. Information about all the page tables is stored in the "Page Directory".
21. The page directory is of 4KB and has 1K "Page Directory Entries" (PDEs) each of size 4 bytes. Each PDE gives information about a Page Table.
22. The PDE has following information:

20 bit page table address : Gives the upper 20 bits of the starting address of the corresponding page table. Lower 12 bits are 0...0 as the page table is of 4 KB and starts from a 4 KB aligned location

D : Dirty bit (explained above)

A : Accessed Bit (explained above)

U/S : User or Supervisor and R/W : Read or Read and Write (explained above)

P : Present bit (explained above)

23. The Page Directory is of 4 KB and begins from a 4 KB aligned location.
24. The address of the page directory is given by the PGBR (page Directory Base Register) field in CR3.
25. The 32 bit Linear Address can be divided into three parts.

The higher 10 bits select one PDE out of 1K PDEs in the page directory.

This gives the starting address in the page table.

The next 10 bits select one PTE out of 1K PTEs in the page table.

This gives the starting address of the page frame.

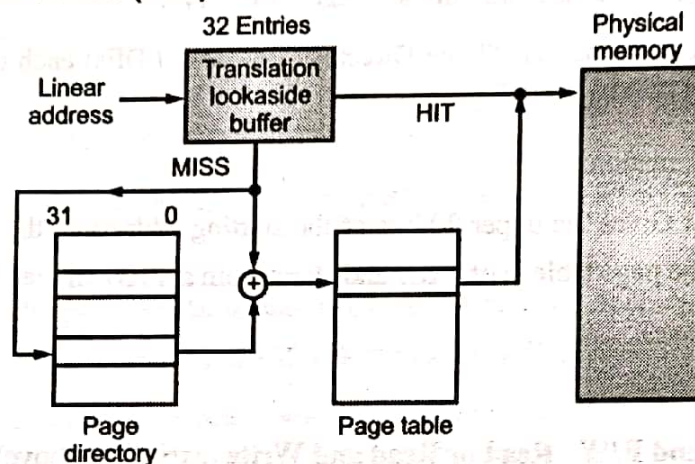
Finally, the lowest 12 bits (offset) select a location within the 4KB page.

26. This means, to access any location, μP must first access a PDE in the page directory then a PTE in the page table, then access the page. This can make the process very slow. To speed up the process a "Translation Look-aside Buffer" is used (called TLB).
27. The TLB is an on chip cache which stores 32 most recently used PTEs and PDEs. This makes subsequent access to these pages (whose information is cached in the TLB) much faster as there is no need to access the Page directory and the page table. μP can directly obtain the starting address of the page frame from the TLB and hence directly access the page. Due to principle of "Locality of reference" most systems get a Hit ratio of >98% on the TLB, thus making the operations very fast.



U/S	R/W	Permitted Level 3	Permitted Access Levels 0, 1 or 2
0	0	None	Read/Write
0	1	None	Read/Write
1	0	Read-Only	Read/Write
1	1	Read/Write	Read/Write

Translation Look-aside Buffer (TLB)



(1F20)Fig. 10.15.4 : TLB

10.16 University Questions and Answers

Dec. 2015

Q. 2(a) Explain memory management in details in 80386DX processor. (Refer section 10.12) (10 Marks)

May 2016

Q. 1(b) Explain V86 mode of 80386DX. (Refer section 10.3.1) (5 Marks)

Dec. 2016

Q. 1(b) Explain flag register of 80386DX. (Refer section 10.3) (5 Marks)

Q. 3(b) Explain data segment descriptor with neat diagram. (Refer section 10.14) (10 Marks)

Q. 6(b) Write short note on : Virtual 86 mode of 80386DX (Refer section 10.3.1) (5 Marks)

Q. 6(d) Write short note on : Control registers of 80386DX (Refer section 10.4) (5 Marks)

**May 2017**

- Q. 1(b) What is GDT? Explain structure of GDT. (Refer section 10.9) (5 Marks)
- Q. 3(b) Explain, with neat diagram, address translation mechanism implemented on 80386 DX. (Refer sections 10.13 and 10.15) (10 Marks)
- Q. 6(b) Write short note on : Virtual 86 mode of 80386 DX (Refer section 10.3.1) (5 Marks)
- Q. 6(d) Write short note on : Control registers of 80386 DX. (Refer section 10.4) (5 Marks)

Dec. 2017

- Q. 1(b) Draw format of selector and explain its field. (Refer section 10.12) (5 Marks)
- Q. 4(b) Draw format of selector and explain its field. (Refer sections 10.13 and 10.15) (5 Marks)
- Q. 5(b) Draw and explain EFLAG register format of 80386 DX. (Refer section 10.3) (10 Marks)

May 2018

- Q. 1(e) Explain Virtual Mode (VM86) 80386 processor (Refer section 10.3.1) (5 Marks)
- Q. 4(b) Explain Segment Descriptor of 80386 Processor (Refer section 10.14) (10 Marks)

Dec. 2018

- Q. 1(c) Explain flag register of 80386 microprocessor. (Refer section 10.3) (5 Marks)
- Q. 4(a) Explain the modes of operation of 80386 microprocessor (Refer section 10.5) (10 Marks)

May 2019

- Q. 1(c) Explain VM, RF, IOPL and NT flags of 80386 microprocessor. (Refer section 10.3) (5 Marks)
- Q. 4(a) Differentiate Real Mode, Protected Mode and virtual 8086 mode of 80386 microprocessor. (Refer sections 10.5 and 10.6) (10 Marks)

Chapter ends...**□□□**



Pentium Processor

11.1	Salient Features of 80586 – Pentium 1	11-3
✓	Syllabus Topic : Pentium Architecture	11-4
11.2	Pentium I Super Scalar Architecture.....	11-4
	Q. Draw and explain block diagram of Pentium processor. (May 17, 10 Marks).....	11-4
✓	Syllabus Topic : Integer Pipeline Stages	11-6
11.3	Pentium I Integer Pipeline Stages	11-6
	Q. Explain in brief, pipeline stages on Pentium processor. (May 16, 5 Marks).....	11-6
	Q. Explain integer pipeline of Pentium processor? (May 17, 5 Marks).....	11-6
11.3.1	Stage 1 : Prefetch	11-6
11.3.2	Stage 2 : Decode	11-6
	Q. Write instruction issue algorithm used in Pentium. (Dec. 17, 5 Marks).....	11-6
	Q. Explain the instructions issue algorithm of Pentium processor (Dec. 18, 10 Marks).....	11-6
	Q. Explain an instruction issue algorithm of Pentium processor. (May 19, 5 Marks).....	11-6
11.3.3	Stage 3 : Decode 2 or Address Generation Stage	11-7
11.3.4	Stage 4 : Execution Stage.....	11-7
11.3.5	Stage 5 : Write-Back stage.....	11-8
✓	Syllabus Topic : Floating Point Pipeline Stages	11-8
11.4	Floating Point Instruction Pipeline Stages	11-8
	Q. Explain in brief, pipeline stages on Pentium processor. (May 16, 5 Marks).....	11-8
	Q. Draw and explain floating point pipeline for Pentium processor. (May 18, 5 Marks).....	11-8
✓	Syllabus Topic : Branch Prediction Logic	11-9



11.5	Pentium I Branch Prediction Logic.....	11-9
Q.	Explain branch prediction logic used in Pentium. (Dec. 15, 10 Marks).....	11-9
Q.	Explain, in brief, branch prediction mechanism is on Pentium processor. (Dec. 16, 10 Marks).....	11-9
Q.	Write short note on : Branch prediction logic. (May 17, 5 Marks).....	11-9
Q.	How flushing problem is minimized in Pentium? Explain. (Dec. 17, 10 Marks).....	11-9
Q.	Explain the branch prediction logic used in Pentium processor (Dec. 18, 10 Marks).....	11-9
✓	Syllabus Topic : Cache Organisation.....	11-11
11.6	Pentium I Cache Memory Organisation.....	11-11
11.6.1	Features of Pentium Cache.....	11-11
✓	Syllabus Topic : MESI Model.....	11-12
11.6.2	The MESI Cache Consistency Model.....	11-12
11.6.3	Code Cache	11-12
Q.	Explain, in brief, cache organization of pentium processor. (May 16, May 19, 10 Marks).....	11-12
Q.	Explain, with neat diagram, cache memory organization is supported by Pentium processor (Dec. 16, May 17, 10 Marks).....	11-12
11.6.4	The Data Cache	11-14
Q.	Explain, in brief, cache organization of pentium processor. (May 16, May 19, 10 Marks).....	11-14
Q.	Explain, with neat diagram, cache memory organization is supported by Pentium processor (Dec. 16, May 17, 10 Marks).....	11-14
Q.	Discuss data cache organization of Pentium. (Dec. 17, 10 Marks).....	11-14
Q.	Explain Data Cache architecture for Pentium processor. (May 18, 10 Marks).....	11-14
11.7	University Questions and Answers	11-17
•	Chapter ends.....	11-17